MTP F26 · TEAM C · RADIO TEAM → PROTOCOLS & SOFTWARE TEAM

Protocol and firmware proposal

From Radio Team (Santiago, Andrian, Marcos) **To** Ibrahim, Guillem, Sofia

Version 0.3.1 · Tue 13 Oct 2026

New in v0.3: all four teams moved to 2.4 GHz. The radio is now the nRF24L01+ (Ebyte E01-ML01SP4, one per box, one omni antenna; decided 13 Oct) at 250 kbps, so **frames are at most 32 B**, each team has its own channel in MRM, and there is no duty-cycle limit any more. The protocol code and the simulator were ported; all 11 scenarios pass.

The radio team has fixed the physical layer: 2.4 GHz, nRF24L01+, 250 kbps, with a tested driver. Everything above it belongs to protocols & software. This is a starting **proposal** built from the competition rules and our link numbers, to speed you up. It is not a decision. Please change whatever you see better.

In short

1. **Compress the file as one deflate stream.** On English text in 100-byte lines it measured **~3×**. The whole file then crosses in **~17 s instead of ~50 s** (model, no loss), which leaves most of the 2 minutes to recover from attacks. The simulator, at 5 % loss, needs 28 s with compression and 73 s without.
2. **Selective-repeat ARQ** with bursts of 16 frames and one bitmap ACK. Frames are delivered strictly in order, because only the **first unbroken run of correct lines** counts.
3. **Resume after any interruption.** A HELLO handshake identifies the file, and the receiver keeps its progress, so an "attack" or a reboot costs seconds, not the round.
4. **Network Mode: gossip, not routing.** Every node rebroadcasts the small file (~20 frames) on a randomised timer (Trickle-style). No addresses or positions are needed, and every node ends up with a copy (that is how teams win NM). It needs agreement with all teams.
5. **Firmware: C on the Pico SDK**, with the radio loop on one core and USB/flash/compression on the other. Build a **PC simulator first** so the protocol is tested before boards exist.

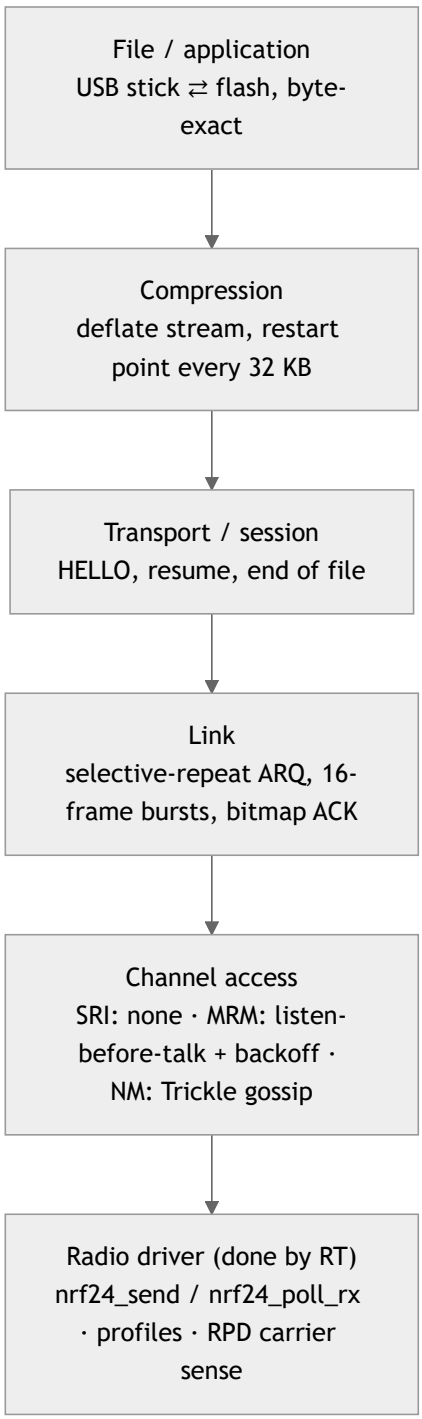
What the rules ask of the software

RULE (COMPETITION PDF)	CONSEQUENCE FOR THE PROTOCOL
File: UNICODE .txt, ~1 MB, ~10 000 lines of equal length, "return"-separated; may contain redundancy	Transfer the file **byte-exact**: no re-encoding, keep the BOM and line endings. "UNICODE" may mean UTF-16, which compresses even better (4.7× measured).
Only identical lines count, and **only the first continuous range** of correct lines	Strict in-order delivery. Never skip a lost frame to send newer data, and stop when the file is complete.
5 min setup while wrapped in foil; the professor then picks the TX device at random; GO/STOP by button; 3 min to upload	Load the file into **both** devices at setup. A role switch (TX/RX) is read when the button is pressed. Save the received file to USB after STOP.

RULE (COMPETITION PDF)	CONSEQUENCE FOR THE PROTOCOL
"The system should be designed to self-recover from eventual interruptions on the link"; "attacks" during the round	Retry forever with backoff, resume by HELLO, watchdog. Received data persists across a reboot.
NM: random order of teams on two rims, 5 lines / 0.5 KB, 5 min; win = deliver a copy from one of your nodes	A flooding/gossip protocol shared by all teams, where every node stores the file. **No switch may indicate the position in the network.**
MRM: all teams transmit at once, 260 m, interference	Channel-access layer (listen before talk, backoff) and a plan agreed with the other teams.
Quick Mode: one line (< 500 chars), zero errors, < 2 min, first attempt only	A simple, robust first version is enough. Reliability beats speed there.
CNAF UN-85 b) (radio team): 2.4 GHz, 10 mW e.i.r.p., no duty-cycle limit; nRF24 frames ≤ 32 B	No transmit-time budget. Headers must stay small: every byte of header is 3 % of a frame.

Protocol stack

The radio team delivers the bottom layer, ready to use. Protocols & software owns the rest.



Frame formats

Each radio frame carries at most **32 bytes** (nRF24 dynamic payload). A full DATA frame takes **1.32 ms** on air at 250 kbps.

`type|flags` 1 B	`session` 1 B	`seq` 2 B	`compressed file bytes` ≤ 28 B	`CRC` 2 B · radio

DATA, SRI/MRM. 32 B at most; the radio adds and checks the CRC (profile TEAM_250K). The 16-bit seq covers files up to 1.83 MB. *session* is a random byte chosen at HELLO, so frames from earlier runs or other teams are dropped.

type|flags	session	next_seq	bitmap	link	CRC
1 B	1 B	2 B	2 B	1 B	2 B · radio

ACK. next_seq = every frame before it has arrived. bitmap bit i = frame next_seq+1+i arrived. The last byte is for link logging: the nRF24 has no RSSI, so it carries the RPD bit (signal above about −76 dBm at the antenna).

type	session	file_len	file_crc32	codec	name	CRC
1 B	1 B	4 B	4 B	1 B	≤ 21 B	2 B · radio

HELLO (sender) and HELLO_ACK (receiver; replaces name with next_seq). They identify the file, so a rebooted receiver can resume where it stopped.

Type values (proposal): 0x01 DATA, 0x02 ACK, 0x03 HELLO, 0x04 HELLO_ACK, 0x05 END, 0x4E NM_DATA. Flags: last-in-burst, retransmission.

Link procedure (SRI and MRM)



Timing

- Burst of 16 × 32 B frames: ≈ **26 ms** (1.32 ms each + 0.3 ms gap)
- ACK + 2 radio turnarounds: ≈ **2.5 ms**
- ACK timeout: ≈ **11.6 ms** at 250 kbps (scales with bit rate), then resend the burst
- Goodput (model, with 15 % software overhead): ≈ **159 kbps**

Resend rules

Each burst first resends every frame the last bitmap marked missing, then fills with new frames. The window never advances past a gap, so the receiver's output is always one continuous run.

After 8 silent timeouts, send HELLO again with backoff. 250 kbps is already the nRF24's slowest rate, so there is no slower fallback. Never give up before STOP.

Recovering from attacks

Jamming or shadowing only looks like lost ACKs, and the retry loop covers it. A reboot of either box is handled by HELLO. The receiver keeps `next_seq` and the data in flash, and the sender re-reads the file from flash.

Hardware watchdog on both cores, plus brown-out reset.

Compression: measured

Measured on 7 Oct with Python's zlib, lzma and bz2. The corpus was the English text of the course rules and slides, laid out as equal 100-byte lines (55 KB, no repetition). The real file is unknown and "may feature redundancy", so treat these as a floor. Re-run on any sample file we get.

METHOD	UTF-8	UTF-16LE	NOTES
deflate, one stream (zlib level 6–9)	2.97×	4.7–4.8×	Recommended. The receiver inflates progressively as frames arrive in order.
deflate, independent 32 KB blocks	2.74×	4.31×	Gives restart points after a receiver reboot. Use a full flush every 32 KB inside the stream for the same effect.
deflate, independent 4 KB blocks	2.23×	3.32×	Only needed if delivery were out of order. It isn't.
deflate, independent 1 KB blocks	1.89×	2.56×	
lzma / bz2, whole file	3.3×	6.3–6.4×	Better ratio, but too much RAM and CPU for the micro. Reference only.

> On the Pico 2, compress during the 5-minute setup: deflate needs roughly 300 KB of RAM `ESTIMATE`, which fits in 520 KB before the radio buffers are allocated. Decompression needs about 45 KB. Library suggestion: miniz (MIT licence, both directions). Measure the speed on the bench and fall back to a lower level if setup time gets tight.

EFFECT PER 2-MIN ROUND (1 MB FILE, 250 KBPS, 159 KBPS GOODPUT)	TIME TO SEND ALL OF IT (MODEL, NO LOSS)	SIMULATOR, 5 % LOSS
No compression	≈ 50 s	73 s
4 KB independent blocks (2.2×)	≈ 23 s	–
deflate stream (3.0×; 2.62× on the repo sample)	≈ 17 s	28 s
deflate stream at 1 Mbps (SRI option)	≈ 5 s	13 s

Network Mode: gossip proposal for all teams

No node knows its position, and the rules forbid a position switch. The file is only about 500 B, or 20 frames. So we suggest that nothing is routed: **every node that holds part of the file rebroadcasts it** on a randomised timer, following the Trickle algorithm (IETF RFC 6206), until STOP.

type 0x4E 1 B	file_id 2 B	idx 1 B	total 1 B	file bytes ≤ 25 B	CRC-16 sw 2 B

NM_DATA: **the inter-team standard** on the NM_COMMON radio profile. file_id = CRC-16 of the whole file. The radio's CRC protects each hop; the software CRC (CCITT-FALSE, big-endian) checks the data end to end across relays.

Behaviour

- The sender (SW corner) starts with the whole file.
- Every node keeps every valid chunk with the same `file_id`. When it hears its first new chunk, it starts rebroadcasting everything it holds.
- Rebroadcast interval: random in [I/2, I], starting at I = 1 s and doubling up to 8 s. Skip a turn if 3 or more identical copies were heard in that interval.
- Once all `total` chunks check out, the node writes the file to flash and saves it to USB after STOP.

Why it fits

- **No addresses or positions**, so it works in any random order of teams.
- **Every node ends up with a copy**, which is the win condition. That makes the +2 bonus, and the +0.5 "poker bonus" for all teams, likely.
- TX time: about 33 ms per full rebroadcast; in the simulator the busiest node transmits about 1.1 s in 5 min. No duty-cycle limit at 2.4 GHz.
- Simple enough for every team to implement on any nRF24L01+ board.

To agree with the other teams

- The frame above, the radio settings (radio team's NM draft) and the timer values.
- An interop day before week 6: every team's box in one room, on the NM channel (RF_CH 80, 2480 MHz).
- Protocols & software leads this with the other teams' protocol people; the radio team handles the radio settings.

MRM: one channel per team

All teams transmit at the same time, 260 m apart. At 2.4 GHz each team gets its own channel, so each keeps its full rate. The radio team's draft for the four teams is RF_CH 24 / 49 / 74 / 82 (D-R4), to be agreed after the campus WiFi survey.

- The risk is a neighbouring team's box a few metres away, which can arrive 40–55 dB above our partner. Channel spacing and distance between boxes handle it (radio brief, MRM section).
- **Listen before talk** with the RPD bit before each burst stays an option if a channel turns out to be shared with WiFi.

Software ask for now: keep the **channel number** and the channel-access policy **configurable at run time**, so the final plan doesn't need a rewrite.

Firmware architecture (Pico 2 suggested to electronics)

MODULE	RUNS ON	JOB	OWNER
`radio/fw/nrf24 + crc16`	core 1	Radio driver, profiles, RPD carrier sense. **Done, 138 host checks.**	RT
`board_hal`	—	SPI, pins, timers for the chosen board	RT + ET

MODULE	RUNS ON	JOB	OWNER
`link`	core 1	Frames, bursts, ACK bitmap, timeouts	PT
`session`	core 1	HELLO / resume / END, role from the switch	PT
`codec`	core 0	miniz deflate at setup; inflate on the receiver	PT
`store`	core 0	Raw flash region (~3 MB) for the file sent and the file received, with a progress record	PT
`usb_msc`	core 0	TinyUSB host + PIO-USB + FatFs: copy the stick's .txt to flash, write the result back	PT
`nm_gossip`	core 1	Trickle timers, chunk store	PT
`ui`	core 0	Button, role and mode switches, LED patterns (the rules require a blinking pilot light)	PT

> Pico pitfall: erasing or writing flash stalls code that runs from flash on **both** cores. Keep the radio loop in RAM (`__not_in_flash_func`), buffer received data in RAM, and write flash in 4 KB sectors between bursts. Test this early, because it is the classic way to lose a round.

Simulator: results on the nRF24 radio updated 9 Oct

The radio team built the simulator in `proto/`. It runs the **same C protocol code** planned for the Pico (link with resume, NM gossip, deflate) against a modelled channel:

- half-duplex radios, with airtime from the real driver;
- collisions, and random or bursty loss;
- a jamming blackout, and reboots.

Scoring copies the judges: decompress, then count the first unbroken run of identical lines. `make -C proto test` runs 11 scenarios in about 1 second.

SCENARIO (1 MB SAMPLE, DEFLATE 2.62×, 250 KBPS, SEED 1)	LINES	WHOLE FILE IN
Clean channel	10 000	24 s
5 % random loss	10 000	28 s
Bursty fades (~6 % loss)	10 000	30 s
10 s jamming attack + 5 % loss	10 000	38 s · back 0.09 s after the attack
Receiver or sender reboot at 15 s, 5 % loss	10 000	28 s
1 Mbps (SRI option), 5 % loss	10 000	13 s
No compression, 5 % loss	10 000	73 s (was 9 643 lines at 868 MHz)
NM, 10 nodes, 10 % / 70 % loss at 1 / 2 hops	all nodes	12 s
NM, 10 nodes, 30 % / 95 % loss	all nodes	43 s

> **It already caught a bug** (on the old 868 MHz radio). With fixed timeouts, the 38.4 kbps fallback delivered only 883 lines. One frame (15.2 ms) outlasted the receiver's 15 ms idle-ACK timer, so the receiver ACKed in the middle of bursts and

went deaf. All timeouts are now derived from the bit rate.

Still open in the simulator: an MRM scenario with four teams on their own channels, listen-before-talk, and loss taken from measured packet-error rates instead of fixed percentages. Assumptions to measure on the bench: 1 ms turnaround, 0.3 ms gap between frames.

Milestones

BY	PROTOCOLS & SOFTWARE	WITH
`Fri 16 Oct`	Agree or amend this proposal; set up a shared git repo; simulator skeleton	RT
`Fri 23 Oct`	Link + session in the simulator; send the NM gossip draft to the other teams	RT (NM radio settings)
`Fri 30 Oct`	On bench boards: USB stick → flash; one line over the air	ET, RT
`Fri 6 Nov`	**Quick Mode attempt** (one error-free line, first try)	all
`Fri 20 Nov`	Compression, full SRI throughput, resume after reboot, NM interop day	RT, other teams
`Fri 27 Nov`	MRM channel access; field tests at 70 m and 260 m	RT
`Fri 4 Dec`	Pretest, fully boxed	all

What we need from protocols & software

1. Agree or counter-propose the stack, frames and compression by **Fri 16 Oct**.
2. An owner for the NM gossip talks with the other teams.
3. A decision between Pico SDK (C, our suggestion) and Arduino-Pico, together with electronics.
4. Two questions for Marcos to add to the Atenea list: is a TX/RX **role switch** allowed (it does not encode network position)? What file name should the received file have on the USB stick?

Related pages: Team C Radio Brief v0.5 (radio settings, MRM channel plan, test plan) · Team C Hardware Proposal v0.4 (Pico 2 + E01-ML01SP4 board). Rules quoted from *MTP F26 – Program Competition Rules – Network Upgrade III + Sustainability*.

References: RFC 6206, The Trickle Algorithm · miniz · Pico-PIO-USB · TinyUSB.